information in mind, you can then look at your code to see where bugs might lie and how you can attempt to determine whether there are any bugs in a particular section.

Perhaps the most obvious and straightforward thing to do is *look at the code*. Although this might seem obvious, it's actually difficult to do in a thorough way. When you read code you've just written, it's all too easy to read what you intended to write rather than what's actually there. Likewise, when reviewing code that others have written, it's tempting to just give it a quick read-through, check it off against your local coding standards, and highlight any glaringly obvious problems. What's needed is to spend the time really going through the code with a fine-tooth comb, thinking about the concurrency issues—and the non-concurrency issues as well. (You might as well, while you're doing it. After all, a bug is a bug.) We'll cover specific things to think about when reviewing code shortly.

Even after thoroughly reviewing your code, you still might have missed some bugs, and in any case you need to confirm that it does indeed work, for peace of mind if nothing else. Consequently, we'll continue on from reviewing the code to a few techniques to employ when testing multithreaded code.

10.2.1. Reviewing code to locate potential bugs

As I've already mentioned, when reviewing multithreaded code to check for concurrency-related bugs, it's important to review it thoroughly, with a fine-tooth comb. If possible, get someone else to review it. Because they haven't written the code, they'll have to think through how it works, and this will help uncover any bugs that may be there. It's important that the reviewer have the time to do the review properly—not a casual two-minute quick glance, but a proper, considered review. Most concurrency bugs require more than a quick glance to spot—they usually rely on subtle timing issues to actually manifest.

If you get one of your colleagues to review the code, they'll be coming at it fresh. They'll therefore see things from a different point of view and may